Guía de Estudio Completa: Redes Neuronales y Perceptrón Multicapa (MLP)

CS/IA — ESPOL-FIEC · Curso de Inteligencia Artificial (Prof. Enrique Peláez, PAO 2026-I)

Elaborada a partir de: anotaciones normalizadas de clase (Clases 06–12), guía maestra del curso, banco de preguntas de exámenes anteriores, y los textos guía Goodfellow (Deep Learning), Hagan (Neural Network Design), Brownlee (Basic Machine Learning) y Luger (AI: Structures and Strategies).

Cómo usar esta guía

Cada sección sigue la misma estructura: **definición formal → notación → derivación matemática → ejemplo numérico resuelto → errores comunes → conexión con preguntas de examen**. Las cajas marcadas como ⚠ **NIVEL EXAMEN** corresponden a temas que el profesor señaló explícitamente como material de evaluación. Las cajas 🔁 **Trampa** describen errores recurrentes en el banco de preguntas.

Índice

1. De la neurona biológica al perceptrón artificial
2. El perceptrón: modelo formal, geometría y compuertas booleanas
3. Funciones de activación: catálogo formal y derivadas
4. La barrera lineal: por qué el perceptrón no resuelve XOR
5. El Perceptrón Multicapa (MLP): arquitectura y notación
6. Dimensiones de las matrices de pesos y el truco del bias
7. MLP como aproximador universal y como función booleana universal
8. Propagación hacia adelante (Forward Propagation)
9. Funciones de salida multiclase: one-hot y softmax
10. Funciones de error / divergencia
11. Aprendizaje: la Regla Delta (Perceptron Learning Rule)
12. Backpropagation: derivación completa por regla de la cadena
13. Notación vectorial general y el jacobiano
14. Estrategias de actualización: online (SGD), batch, mini-batch
15. Regularización en MLP: overfitting, dropout, L1/L2
16. Banco de problemas resueltos (estilo examen)
17. Tabla-resumen de fórmulas críticas
18. Checklist final de dominio

1. De la neurona biológica al perceptrón artificial

1.1 Motivación histórica

El modelo de neurona artificial fue introducido por **McCulloch y Pitts (1943)** como una abstracción matemática de la neurona biológica. Esta analogía es la piedra angular de todo el conexionismo:

Componente biológico	Componente artificial	Función
Dendritas	Entradas $x_1, x_2, \ldots, x_n$	Reciben señales
Sinapsis	Pesos $w_1, w_2, \ldots, w_n$	Ponderan la intensidad de cada señal
Soma (cuerpo celular)	Unidad de activación / suma ponderada + función f	Integra las señales y decide si se "dispara"
Axón	Salida y o a	Transmite el resultado a la siguiente neurona
Umbral de disparo	Umbral θ o bias b	Determina la sensibilidad de la neurona

Idea central del curso (repetida en clases 06, 09 y 10): toda la familia de modelos — regresión lineal → regresión logística → perceptrón → MLP → CNN → RNN — es, en esencia, la misma función

$$y = f(\mathbf{W}\mathbf{X} + b)$$

optimizada mediante **gradiente descendente** y **regla de la cadena**. Lo único que cambia entre arquitecturas es (a) la estructura de la conexión (densa, convolucional, recurrente) y (b) la forma del dato de entrada (escalar, vector, volumen, secuencia). Comprender esta unidad estructural es la clave para no perderse en la notación de cada arquitectura.

1.2 El "artilugio" del bias (bias trick)

⚠ **NIVEL EXAMEN.** Toda neurona calcula, en general,

$$z = \sum_{j=1}^{n} w_j x_j + b$$

Para simplificar la notación (y para que las derivadas de backpropagation traten al bias exactamente igual que a un peso), se define una entrada ficticia $x_0 = 1$ con peso $w_0 = b$. De este modo:

$$z = \sum_{j=0}^{n} w_j x_j = \mathbf{W}^T \mathbf{X}, \qquad \text{con } x_0 \equiv 1,\ w_0 \equiv b$$

🔁 **Trampa recurrente:** en casi todas las preguntas de examen sobre dimensiones de matrices de pesos, o sobre número de parámetros, el error más común es *olvidar el bias*. Recordar siempre: **cada neurona añade exactamente un parámetro de bias**, independientemente de cuántas entradas tenga.

---

2. El perceptrón: modelo formal, geometría y compuertas booleanas

2.1 Definición formal

El **perceptrón** (Rosenblatt, 1958) es la unidad computacional más simple: calcula una combinación lineal de sus entradas y le aplica una función de activación (originalmente un escalón).

$$z = \sum_{j=0}^{n} w_j x_j = \mathbf{W}^T \mathbf{X}$$

$$a = f(z)$$

Modelo con función escalón (umbral duro):

$$a = \begin{cases} 1 & \text{si } \mathbf{W}^T \mathbf{X} \geq \theta \\ 0 & \text{si } \mathbf{W}^T \mathbf{X} < \theta \end{cases}$$

2.2 Interpretación geométrica

Un perceptrón con activación tipo escalón define un **hiperplano** en el espacio de entradas:

$$\mathbf{W}^T \mathbf{X} - \theta = 0$$

Este hiperplano separa el espacio en dos semiespacios: uno donde la neurona se activa ($a = 1$) y otro donde no ($a = 0$). En $\mathbb{R}^2$ este hiperplano es una **recta**. Esta es la propiedad que el profesor resume con la frase mnemotécnica:

> **"Una neurona sólo reconoce líneas."**

Es decir: **un perceptrón únicamente puede resolver problemas linealmente separables** (aquellos en los que existe una recta —o hiperplano, en dimensiones mayores— que separa perfectamente las clases).

2.3 Ejemplo resuelto: compuerta AND con un perceptrón

Enunciado. Diseñar un perceptrón con función escalón que implemente la compuerta lógica AND.

Solución. Sean $w_1 = 1, w_2 = 1, \theta = 1.5$. Se evalúa $z = x_1 w_1 + x_2 w_2$ para las 4 combinaciones:

x_1	x_2	$z = x_1 + x_2$	$z \geq 1.5$?	**Salida**	**AND esperado**
0	0	0	No	0	0
0	1	1	No	0	0
1	0	1	No	0	0
1	1	2	**Sí**	**1**	1

Solo la combinación $(1, 1)$ supera el umbral 1.5, replicando exactamente la tabla de verdad de AND. **Geométricamente**, la recta $x_1 + x_2 = 1.5$ separa el punto $(1, 1)$ de los otros tres.

De manera análoga se construyen OR (con $\theta = 0.5$, mismos pesos) y NOT (con un solo peso negativo).

2.4 Funciones de transferencia clásicas (nomenclatura de Hagan / MATLAB)

Nombre	**Fórmula**	**Rango**	**Uso típico**
hardlim	$1[z \geq 0]$ (escalón 0/1)	$\{0, 1\}$	Perceptrón clásico
hardlims	escalón simétrico	$\{-1, 1\}$	Perceptrón bipolar
purelin	$f(z) = z$	$\mathbb{R}$	Capa de salida en regresión
logsig	$\dfrac{1}{1 + e^{-z}}$	$(0, 1)$	Clasificación binaria

🔁 **Trampa:** la función de activación (o de transferencia/umbral) es un **hiperparámetro**: se elige según el problema, no es intrínseca al modelo. No confundir el **umbral** θ (un escalar de decisión) con el **bias** b (un parámetro entrenable); en la práctica ambos cumplen roles equivalentes una vez aplicado el truco del bias (θ se reescribe como $-b$).

---

3. Funciones de activación: catálogo formal y derivadas

⚠ **NIVEL EXAMEN.** Las funciones de activación deben ser (salvo el escalón clásico) **diferenciables**, porque backpropagation depende de la regla de la cadena, y ésta exige que cada función involucrada tenga derivada bien definida.

Función	**Fórmula** $f(z)$	**Derivada** $f'(z)$	**Rango**	**Uso típico**	**Observaciones / trampa**
Lineal	z	1	$\mathbb{R}$	Regresión, capa de salida continua	No introduce no linealidad: componer capas lineales equivale a una sola capa lineal
Escalón (hardlim)	$1[z \geq \theta]$	0 (c.t.p.)	$\{0, 1\}$	Perceptrón clásico	**No diferenciable** → no sirve para backpropagation estándar
Sigmoide (logsig)	$\frac{1}{1+e^{-z}}$	$f(z)\,(1-f(z))$	$(0, 1)$	Clasificación binaria, probabilidad	Satura para $
Tangente hiperbólica (tanh)	$\frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	$1-f(z)^2$	$(-1, 1)$	Capas ocultas en RNN	También satura en los extremos
ReLU	$\max(0, z)$	$\begin{cases} 1 & z>0 \\ 0 & z<0 \end{cases}$	$[0, \infty)$	Capas ocultas en MLP/CNN profundas	No derivable en $z=0$ (se define por convención); "neuronas muertas" si $z<0$ siempre
Softmax	$\frac{e^{z_k}}{\sum_j e^{z_j}}$	(jacobiano, ver §9)	distribución de probabilidad (suma 1)	Capa de salida multiclase	**Nunca** se usa como activación de capa oculta; no confundir con sigmoide en problemas multiclase

Derivación de la derivada de la sigmoide (aparece explícitamente marcada por el profesor como pregunta de examen):

$$f(z) = \frac{1}{1+e^{-z}}$$

$$f'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \frac{1}{1+e^{-z}} \left(1 - \frac{1}{1+e^{-z}}\right) = f(z)\,(1 - f(z))$$

Esta forma es muy usada en la práctica porque permite calcular la derivada **reutilizando el valor ya calculado en el forward pass** ($f(z)$), sin recomputar la exponencial — la misma lógica que sustenta la eficiencia de backpropagation.

🔁 **Trampa (Anotaciones, Clase 09):** *no* existe derivada respecto de x en la **capa de entrada**, porque ahí x son datos (constantes), no parámetros. Las derivadas de backpropagation con respecto a la "entrada" de una capa oculta sí existen, porque esa entrada es en realidad la salida y de la capa anterior, que sí depende de parámetros entrenables.

---

4. La barrera lineal: por qué el perceptrón no resuelve XOR

4.1 El problema

Tabla de verdad de XOR:

x_1	x_2	**XOR**
0	0	0
0	1	1
1	0	1
1	1	0

Teorema (informal). No existe ningún par (w_1, w_2, θ) tal que un único perceptrón con activación escalón separe correctamente los puntos $\{(0,1),(1,0)\}$ (salida 1) de $\{(0,0),(1,1)\}$ (salida 0).

Demostración por contradicción. Si tal perceptrón existiera, se cumplirían simultáneamente las cuatro desigualdades:

$$0 < \theta \quad (\text{de } (0,0) \to 0)$$

$$w_1 \geq \theta \quad (\text{de } (1,0) \to 1)$$

$$w_2 \geq \theta \quad (\text{de } (0,1) \to 1)$$

$$w_1 + w_2 < \theta \quad (\text{de } (1,1) \to 0)$$

Sumando las dos desigualdades centrales: $w_1 + w_2 \geq 2\theta$. Como $\theta > 0$, se tiene $w_1 + w_2 \geq 2\theta > \theta$, lo cual **contradice** directamente $w_1 + w_2 < \theta$. ■

Geométricamente: los cuatro puntos de XOR forman un cuadrado donde las clases están ubicadas en **vértices opuestos (diagonales)**; ninguna recta puede separar diagonales de un cuadrado. XOR **no es linealmente separable**.

4.2 La solución: composición de fronteras lineales

$$XOR(x_1, x_2) = (x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge x_2)$$

$$XNOR(x_1, x_2) = (x_1 \wedge x_2) \vee (\neg x_1 \wedge \neg x_2)$$

Cada término de la disyunción es en sí mismo linealmente separable, y por tanto puede implementarse con **una neurona**. La disyunción final combina esas dos fronteras con **una tercera neurona**. Esto exige una **capa oculta**: de aquí nace la necesidad estructural del **perceptrón multicapa (MLP)**.

4.3 Ejemplo resuelto: red que reconoce $(x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge x_3)$

(Pregunta de examen recurrente, estilo Preguntas 27 y 71 del banco.) Con activación *hardlim* de umbral ≥ 2 y entradas binarias, una arquitectura típica de 2 neuronas ocultas + 1 de salida es:

- Neurona oculta 1: $a_1^{(2)} = \text{hardlim}(1 + x_1 - x_2 \,;\, \theta = 2)$ → detecta $x_1 \wedge \neg x_2$
- Neurona oculta 2: $a_2^{(2)} = \text{hardlim}(1 - x_1 + x_3 \,;\, \theta = 2)$ → detecta $\neg x_1 \wedge x_3$
- Neurona de salida: $a_1^{(3)} = \text{hardlim}(a_1^{(2)} + a_2^{(2)} \,;\, \theta = 1)$ → OR de las dos anteriores

Este patrón —"cada neurona oculta detecta una condición parcial (un término de un mintérmino/conjunción), la neurona de salida las combina mediante disyunción"— es la plantilla general para construir **cualquier función booleana** con una MLP de una sola capa oculta, lo cual conecta directamente con la siguiente sección.

🔁 **Trampa:** en estas preguntas de "seleccione los pesos correctos" hay frecuentemente **más de una solución válida**, porque los pesos no son únicos (ver §7.3, *no unicidad de los parámetros*). Se debe verificar la respuesta evaluando **las 4 (u 8) combinaciones de entrada**, no solo confiar en la intuición.

---

5. El Perceptrón Multicapa (MLP): arquitectura y notación

5.1 Definición formal

Un **Perceptrón Multicapa (MLP)** es una red neuronal **feedforward** (la información fluye únicamente hacia adelante, sin ciclos ni saltos) y **densa** (cada neurona de una capa se conecta con **todas** las neuronas de la capa siguiente — en Keras: capa `Dense`; en PyTorch: `Linear`/*fully connected*).

Consta de tres tipos de capa:

1. **Capa de entrada:** no contiene neuronas propiamente dichas; solo aloja el vector de características **x**.
2. **Capas ocultas** (1 o más): "donde ocurre la magia" — cada neurona combina linealmente las salidas de la capa anterior y aplica una activación no lineal.
3. **Capa de salida:** produce la predicción final (una neurona para regresión/binaria, K neuronas con softmax para multiclase).

Profundidad de la red = la distancia (número de capas) más larga entre la entrada y la salida.

5.2 Notación estándar por capas (superíndices y subíndices)

> *"El secreto de todo está en cómo administras los superíndices y subíndices."* — El superíndice indica **la capa**; el subíndice indica **la neurona destino** dentro de esa capa.

Para la capa k:

$$z_j^{(k)} = \sum_i w_{ij}^{(k)}\, y_i^{(k-1)} \qquad\qquad y_j^{(k)} = f_k\left(z_j^{(k)}\right)$$

donde:

- $z_j^{(k)}$: entrada neta (pre-activación) de la neurona j en la capa k.
- $w_{ij}^{(k)}$: peso que conecta la neurona i de la capa $k-1$ con la neurona j de la capa k.
- $y_i^{(k-1)}$: salida (activación) de la neurona i en la capa anterior. Por convención, $y_i^{(0)} = x_i$ (la capa 0 es la entrada cruda).
- $f_k(\cdot)$: función de activación de la capa k (un hiperparámetro **por capa**, no por neurona individual — usar activaciones distintas por neurona dentro de la misma capa desestabiliza el cálculo del gradiente).

5.3 Por qué se necesita no linealidad

Si todas las capas usaran activación lineal ($f(z) = z$), la composición de L transformaciones afines seguiría siendo una única transformación afín:

$$y = W^{(L)}\left(W^{(L-1)}\left(\cdots\left(W^{(1)}x + b^{(1)}\right)\cdots\right) + b^{(L-1)}\right) + b^{(L)} = W_{\text{eq}}\,x + b_{\text{eq}}$$

Es decir, **sin activaciones no lineales, una MLP de cualquier profundidad colapsa a un modelo lineal equivalente a una sola capa**, perdiendo toda capacidad expresiva adicional. Esta es la razón formal de por qué la no linealidad es indispensable para separar clases como XOR.

---

6. Dimensiones de las matrices de pesos y el truco del bias

⚠ NIVEL EXAMEN — tema de altísima frecuencia en el banco de preguntas (10–13, 27–30, 61, 114, 139, 144).

6.1 Regla general

Si la capa k tiene L_k neuronas y la capa siguiente $k+1$ tiene L_{k+1} neuronas, la matriz de pesos que las conecta tiene dimensión:

$$\mathbf{W}^{(k)} \in \mathbb{R}^{\,L_{k+1}\times(L_k+1)}$$

- **Filas** = número de neuronas de la capa **destino** (a la que se llega).
- **Columnas** = número de neuronas de la capa **origen, más uno** (el $+1$ es el bias de cada neurona destino).

Equivalentemente, con bias separado en un vector aparte:

$$\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}, \qquad \mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$$

Ambas convenciones son correctas; la clave del examen es **identificar cuál usa la figura dada** (si el bias se dibuja como una entrada extra "+1" conectada a cada neurona, se usa la primera forma con el $+1$ en las columnas).

6.2 Ejemplo resuelto: conteo de parámetros

Enunciado. Una MLP tiene 2 entradas, una capa oculta de 5 neuronas y una capa de salida de 3 neuronas. ¿Cuántos parámetros tiene la red?

Solución.

Capa oculta ($2 \to 5$): $\quad W^{(1)} \in \mathbb{R}^{5\times(2+1)} \Rightarrow 5 \times 3 = 15$ parámetros.

Capa de salida ($5 \to 3$): $\quad W^{(2)} \in \mathbb{R}^{3\times(5+1)} \Rightarrow 3 \times 6 = 18$ parámetros.

Total: $15 + 18 = 33$ parámetros.

6.3 Ejemplo resuelto tipo examen — imagen aplanada a MLP densa

Enunciado (Pregunta 30/91/140 del banco, variantes). Imagen a color de 400×400 píxeles (sin usar CNN), conectada completamente a una primera capa oculta de 100 neuronas. ¿Cuántos parámetros tiene esa capa?

Solución.

Número de entradas (aplanando la imagen RGB): $400 \times 400 \times 3 = 480{,}000$.

Parámetros $= 100 \times (480{,}000 + 1) = 100 \times 480{,}001 = 48{,}000{,}100$.

(Nota: la respuesta correcta del banco para esta variante es **48,000,100** — confirma que se debe sumar exactamente **un** bias por neurona de destino, no por canal ni por píxel.)

6.4 Ejemplo resuelto: identificar dimensión de $W^{(2)}$ dada una figura

Enunciado (Pregunta 28/114/139). Dada una red con capa de entrada de 2 neuronas, capa oculta de 5 neuronas y capa de salida de 1 neurona, ¿cuál es la dimensión de $W^{(2)}$ (pesos entre la capa oculta y la de salida)?

Solución. Aplicando la regla $\mathbb{R}^{L_{k+1} \times (L_k + 1)}$ con $L_k = 5$ (oculta) y $L_{k+1} = 1$ (salida):

$$W^{(2)} \in \mathbb{R}^{1 \times (5+1)} = \mathbb{R}^{1 \times 6}$$

Es decir, dimensión **1×6** (respuesta correcta del banco para esta variante concreta). 🔁 **Trampa:** en variantes con más capas (ej. Pregunta 139, red de 4 capas), hay que verificar cuidadosamente cuál superíndice corresponde a cuál transición según la numeración de la figura — algunos ejercicios enumeran las *capas de neuronas* (1,2,3,4) y otros enumeran las *matrices de pesos* (1,2,3) empezando en la primera transición.

6.5 No unicidad de los parámetros

⚠ **NIVEL EXAMEN** (Lección 2, pregunta de "modelos con neuronas ocultas reordenadas"). El orden de las neuronas dentro de una capa oculta es **intercambiable**: si se permutan simultáneamente (a) las filas de $W^{(k)}$ correspondientes a esas neuronas y (b) las columnas correspondientes de $W^{(k+1)}$, la red calcula exactamente la misma función. Por tanto:

- **Los pesos entrenados no son únicos** — distintas inicializaciones aleatorias (distinto *seed*) pueden converger a soluciones funcionalmente equivalentes pero numéricamente distintas.
- Esto es precisamente por qué **fijar el seed** es importante para poder **replicar** un experimento exacto.

🔁 **Trampa:** no confundir "no unicidad de los parámetros" con "el gradiente descendente no converge" — son fenómenos distintos: la red puede converger perfectamente y aun así llegar a un conjunto de pesos distinto en cada corrida.

---

7. MLP como aproximador universal y como función booleana universal

7.1 Teorema de aproximación universal (enunciado informal)

Definición formal (enunciado del curso). Una red neuronal *feedforward* con al menos una capa oculta y una función de activación no lineal adecuada (sigmoide, tanh, ReLU, etc.) puede aproximar **cualquier función continua** sobre un dominio compacto con la precisión que se desee, siempre que se disponga de suficientes neuronas ocultas.

Consecuencias que el profesor enfatiza:

1. Las MLP pueden modelar **cualquier función booleana** (por eso se las llama *"funciones booleanas universales" — "podríamos construir un computador con esto"*).
2. Pueden aproximar **cualquier frontera de clasificación**, no solo lineales.
3. Pueden aproximar **funciones de valores continuos** (regresión no lineal, ej. "predecir el clima de mañana").

Intuición geométrica: cada neurona oculta con activación tipo escalón/sigmoide aporta una "media línea" o frontera parcial; al combinar suficientes de estas fronteras parciales en las capas siguientes, se puede aproximar cualquier región del espacio, sin importar su forma.

7.2 Construcción explícita: MLP para XNOR

Enunciado. Diseñar una MLP de 3 capas (entrada, 1 oculta con 2 neuronas, salida con 1 neurona) que calcule XNOR.

Solución (usando $XNOR = (x_1 \wedge x_2) \vee (\neg x_1 \wedge \neg x_2)$):

- Neurona oculta h_1 (detecta $x_1 \wedge x_2$): pesos $w_1 = 1,\ w_2 = 1$, umbral $\theta = 1.5$ (activación escalón).
- Neurona oculta h_2 (detecta $\neg x_1 \wedge \neg x_2$): pesos $w_1 = -1,\ w_2 = -1$, umbral $\theta = -1.5$ (equivalente a $-x_1 - x_2 \geq -1.5 \Leftrightarrow x_1 + x_2 \leq 1.5$, es decir, ambos en 0).
- Neurona de salida (OR de h_1, h_2): pesos $w_{h_1} = 1,\ w_{h_2} = 1$, umbral $\theta = 0.5$.

Verificación con las 4 combinaciones:

x_1	x_2	h_1	h_2	**salida**	**XNOR esperado**
0	0	0	1	1	1
0	1	0	0	0	0
1	0	0	0	0	0
1	1	1	0	1	1

Coincide exactamente. Esta plantilla ("una neurona oculta por mintérmino relevante + una neurona de salida que aplica OR") es generalizable a **cualquier función booleana**, lo que constituye la demostración constructiva de la universalidad booleana de la MLP.

7.3 Aclaración conceptual importante

Una red con muchas capas ocultas **sigue siendo, técnicamente, una MLP**, siempre que sea densa y *feedforward*. El término "red profunda" (*deep network*) suele reservarse informalmente para arquitecturas con muchas capas y, típicamente, con estructura adicional (convoluciones, mecanismos de atención, etc.), pero no existe una frontera numérica estricta y universalmente aceptada entre "MLP" y "red profunda" — es más una cuestión de convención práctica que una definición formal.

---

8. Propagación hacia adelante (Forward Propagation)

8.1 Definición formal

Forward propagation es el proceso de calcular la salida de la red a partir de una entrada dada, recorriendo las capas en orden creciente y aplicando, en cada una, una transformación afín seguida de una activación no lineal.

Forma escalar (por neurona), recursiva sobre capas k y unidades i, j:

$$y_i^{(0)} = x_i \qquad (\text{la capa 0 es la entrada cruda})$$

$$z_j^{(k)} = \sum_i w_{ij}^{(k)}\, y_i^{(k-1)} \qquad y_j^{(k)} = f_k\left(z_j^{(k)}\right), \qquad k = 1, \ldots, n$$

Forma vectorial (por capa), la que se usa en implementación:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\mathbf{a}^{(l)} = f^{(l)}\left(\mathbf{z}^{(l)}\right)$$

con $\mathbf{a}^{(0)} = \mathbf{x}$ y la salida final de la red $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$ (capa L = última capa).

⚠ **Observación de examen:** "solo las neuronas tienen la cantidad z" — la capa de entrada no calcula ningún z, simplemente contiene el vector $\mathbf{x}$.

8.2 Ejemplo numérico resuelto: forward pass en una red pequeña

Enunciado. Sea una red con 2 entradas $x_1 = 1, x_2 = 0$, una capa oculta con 2 neuronas

sigmoides con pesos $W^{(1)} = \begin{bmatrix} 1 & -1 & 0.5 \\ -1 & 1 & 0.5 \end{bmatrix}$ (última columna = bias) y una neurona de salida lineal con pesos $W^{(2)} = \begin{bmatrix} 1 & 1 & -0.5 \end{bmatrix}$. Calcular la salida.

Solución.

Capa oculta:

$$z_1^{(1)} = 1(1) + (-1)(0) + 0.5 = 1.5 \qquad a_1^{(1)} = \sigma(1.5) = \frac{1}{1+e^{-1.5}} \approx 0.8176$$

$$z_2^{(1)} = -1(1) + 1(0) + 0.5 = -0.5 \qquad a_2^{(1)} = \sigma(-0.5) = \frac{1}{1+e^{0.5}} \approx 0.3775$$

Capa de salida (lineal):

$$z^{(2)} = 1(0.8176) + 1(0.3775) - 0.5 = 0.6951$$

$$\hat{y} = f_{\text{lineal}}(0.6951) = 0.6951$$

Este mismo procedimiento —calcular z, aplicar f, pasar el resultado como entrada de la siguiente capa— es exactamente el que se pide en las preguntas de RNN "desenrolladas" del banco de examen (78, 79, 101), donde además interviene la retroalimentación temporal W_{hh}.

8.3 Errores comunes en forward propagation

- Aplicar la activación de la capa de salida a las capas ocultas (o viceversa) — cada capa puede tener una función de activación **distinta**, definida como hiperparámetro.
- Olvidar el bias en el cálculo de z.
- Confundir la dirección de la multiplicación matricial: $\mathbf{W}^{(l)}$ multiplica a $\mathbf{a}^{(l-1)}$ (vector columna), no al revés.

---

9. Funciones de salida multiclase: one-hot y softmax

9.1 Codificación one-hot

Para un problema de clasificación con K clases, cada etiqueta se representa como un vector $\mathbf{d} \in \{0,1\}^K$ con un único componente en 1 (la clase correcta) y el resto en 0. Ejemplo: si las clases son {perro, gato, ave, pez, otro} y el ejemplo es "perro":

$$\mathbf{d} = [1, 0, 0, 0, 0]^T$$

La salida de la red $\hat{\mathbf{y}}$ debe compararse **componente a componente** con $\mathbf{d}$. Es indispensable entrenar con ejemplos **positivos y negativos, balanceados entre clases**; con desbalance leve (p. ej. 10%) el efecto sobre el aprendizaje es despreciable, pero con desbalance severo el modelo aprende a favorecer la clase mayoritaria.

9.2 Softmax

Definición formal. Dada la pre-activación de la capa de salida $\mathbf{z} = (z_1, \ldots, z_K)$, softmax produce una distribución de probabilidad:

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \ldots, K$$

Propiedades:

- $\text{softmax}(\mathbf{z})_k \in (0, 1)$ para todo k.
- $\sum_{k=1}^{K} \text{softmax}(\mathbf{z})_k = 1$ (es una distribución de probabilidad válida).
- La clase predicha es $\hat{c} = \arg\max_k \text{softmax}(\mathbf{z})_k$, que por monotonía del exponencial equivale a $\arg\max_k z_k$.

🔁 **Trampa central del banco (Preguntas 45, 60, 146):** softmax se usa **exclusivamente en la capa de salida** para problemas **multiclase**; nunca reemplaza a la sigmoide en clasificación binaria simple, y nunca se usa como activación de capa oculta. Además, la salida de softmax **no** garantiza que la suma de las activaciones al cubo, al cuadrado, etc. (transformaciones posteriores) sea igual a 1 — solo la salida softmax cruda suma exactamente 1 (Pregunta 60d, 137d: **falso** que $a_1^{(3)} + a_2^{(3)2} + a_3^{(3)3} = 1$ en general).

9.3 Jacobiano de softmax (para backpropagation)

$$\begin{cases} \text{softmax}(\mathbf{z})_k \left(1 - \text{softmax}(\mathbf{z})_k\right) & \text{si } i = k \\ -\,\text{softmax}(\mathbf{z})_k \, \text{softmax}(\mathbf{z})_i & \text{si } i \neq k \end{cases}$$

Esta matriz **no es diagonal** en general (a diferencia de la derivada de la sigmoide, que es "elemento a elemento"), porque cada salida de softmax depende de **todas** las pre-activaciones z_j, no solo de la suya propia. Cuando softmax se combina con la pérdida de entropía cruzada, sin embargo, el gradiente combinado se simplifica notablemente (ver §10.3).

---

10. Funciones de error / divergencia

10.1 Definición general

La función de **error** (o **divergencia**, o **función de costo**) mide cuán lejos está la predicción $\hat{\mathbf{y}}$

del valor deseado $\mathbf{d}$. Se promedia sobre los m ejemplos de entrenamiento:

$$\text{Error}(\mathbf{w}) = \frac{1}{m}\sum_{i=1}^{m} \text{div}(f(\mathbf{x}_i;\mathbf{w}),\, \mathbf{d}_i)$$

donde div es la función de divergencia elegida (**hiperparámetro** del modelo — otra decisión de diseño, igual que la activación).

⚠ **Importante:** la función de divergencia **no forma parte de la arquitectura de la red**; se mide "afuera" de la red, comparando la salida con la etiqueta. Es el punto de partida (no un componente) del proceso de backpropagation.

10.2 Catálogo de funciones de divergencia

Función	Fórmula	Uso típico
Error cuadrático medio (MSE) / distancia euclídea al cuadrado	$\frac{1}{2}\|\hat{\mathbf{y}} - \mathbf{d}\|^2 = \frac{1}{2}\sum_k (\hat{y}_k - d_k)^2$	Regresión
Entropía (binaria)	$-\left[d \log \hat{y} + (1-d)\log(1-\hat{y})\right]$	Clasificación binaria
Entropía cruzada categórica	$-\sum_{k=1}^{K} d_k \log \hat{y}_k$	Clasificación multiclase (con softmax)

La **entropía cruzada** es preferida frente a MSE en clasificación porque penaliza con mucha más fuerza las predicciones **confiadas y erróneas** — lo cual conecta directamente con el ejemplo de la Lección 2 del curso (ver §10.4).

10.3 Derivada combinada softmax + entropía cruzada

Cuando la capa de salida usa softmax y la pérdida es entropía cruzada categórica, el gradiente de la pérdida J respecto de la pre-activación z_k se simplifica extraordinariamente:

$$\frac{\partial J}{\partial z_k} = \hat{y}_k - d_k$$

Esta simplificación es la razón práctica por la cual casi todos los *frameworks* combinan softmax + cross-entropy en una sola operación numéricamente estable, y es también la forma más natural de **inicializar el gradiente en la capa de salida** al comenzar backpropagation (ver §12).

10.4 Ejemplo resuelto: por qué la entropía cruzada penaliza la confianza equivocada

Enunciado (adaptado del ejercicio de la Lección 2 del curso). Se tienen 4 ejemplos con etiqueta y predicción:

Ejemplo	y (verdadero)	h (predicho)
1	1	0.59
2	1	0.01
3	0	0.49
4	0	0.99

Solución. Aplicando costo $= -\log(h)$ si $y = 1$, o $-\log(1-h)$ si $y = 0$:

- Ejemplo 1: $-\log(0.59) \approx 0.53$ (costo moderado-bajo, correcto con confianza media).
- Ejemplo 2: $-\log(0.01) \approx 4.61$ (**costo muy alto**: la red predijo casi 0 cuando la verdad era 1).
- Ejemplo 3: $-\log(1-0.49) = -\log(0.51) \approx 0.67$ (costo moderado, cerca del límite de decisión).
- Ejemplo 4: $-\log(1-0.99) = -\log(0.01) \approx 4.61$ (**costo muy alto**: predijo casi 1 con verdad 0).

Conclusión (respuesta modelo): ningún caso tiene costo exactamente 0 (porque ninguna predicción es exacta), y los casos 2 y 4 muestran el comportamiento característico de la entropía cruzada: **penaliza fuertemente las predicciones hechas con alta confianza cuando resultan erróneas**, mucho más de lo que lo haría el error cuadrático medio.

🔁 **Trampa (Pregunta 62):** con la función de costo logístico dado, si $h_w(x) = y$ entonces el costo es exactamente 0 (para $y = 0$ y $y = 1$); si $y = 0$, el costo tiende a ∞ cuando $h_w(x) \to 1$ (no cuando $h_w(x) \to 0$); y si $h_w(x) = 0.5$ el costo es **estrictamente positivo** para cualquier valor de y (no es cero en el punto de máxima incertidumbre).

---

11. Aprendizaje: la Regla Delta (Perceptron Learning Rule)

11.1 Definición formal

La **regla delta** es la regla de actualización de pesos más simple: ajusta cada peso de manera **proporcional al error** cometido y **a la entrada** que participó en la conexión.

$$\Delta w_{ij} = \alpha \, (t_j - y_j) \, y_i$$

$$w_{ij}^{\text{nuevo}} = w_{ij}^{\text{anterior}} + \Delta w_{ij}$$

Variables:

- $\alpha \in (0, 1)$: tasa de aprendizaje.
- t_j: salida deseada (*target*) de la neurona j.
- y_j: salida calculada de la neurona j.
- y_i: salida de la neurona i que alimenta a j (es decir, la "entrada" a esa conexión particular).

⚠ **Matiz crítico señalado por el profesor:** *"en algunos momentos usamos x y en otros usamos y"*. Si la conexión parte directamente de la **capa de entrada** (capa 0), se usa x_i (el dato crudo). Si la conexión parte de una **capa oculta**, se usa y_i (la salida/activación de la neurona anterior, que es lo que realmente "entra" a la conexión). Formalmente ambas notaciones son consistentes porque $y_i^{(0)} \equiv x_i$.

11.2 Intuición

Si la salida calculada y_j es **menor** que la deseada t_j (el error $t_j - y_j > 0$), se **incrementan** los pesos asociados a entradas activas (positivas), para que la próxima vez la suma ponderada sea mayor. Si y_j es **mayor** que t_j, se reducen esos pesos. La regla delta es el precursor conceptual e histórico directo del gradiente descendente y de backpropagation.

11.3 Algoritmo de entrenamiento del perceptrón

Entrada: conjunto de ejemplos $(\mathbf{x}, t)$, tasa de aprendizaje α, umbral θ. **Salida:** pesos $\mathbf{w}$ que clasifican correctamente todos los ejemplos (si son linealmente separables).

1. Elegir α.
2. Inicializar los pesos (aleatoriamente, normalizados, o mediante *transfer learning*).
3. **Mientras** existan ejemplos mal clasificados: a. Presentar un ejemplo $(\mathbf{x}, t)$ a la red. b. Si la salida calculada difiere del *target*, actualizar: $w^{\text{nuevo}} = w^{\text{anterior}} + \alpha(t - y)x$.
4. **Criterio de parada:** cuando $\Delta w = 0$ para **todos** los ejemplos en una época completa (es decir, ningún ejemplo produce ya corrección).

Teorema de convergencia del perceptrón (Rosenblatt). Si los datos de entrenamiento son linealmente separables, el algoritmo de la regla delta converge a una solución en un número finito de pasos. Si los datos **no** son linealmente separables (como XOR), el algoritmo **nunca converge** — oscila indefinidamente.

11.4 Ejemplo numérico completo resuelto: entrenamiento de un perceptrón AND

Enunciado (ejemplo íntegro de clase). Parámetros: $\alpha = 0.3$, umbral $\theta = 1.5$ (activación escalón), pesos iniciales $w_1 = 0,\ w_2 = 0.25$ (bias omitido por simplicidad pedagógica — *"siempre está"*, indica el profesor). Entrenar sobre la compuerta AND, presentando repetidamente los 4 pares $(x_1, x_2) \to t$.

Solución, época por época (mostrando solo los pasos donde hay error, es decir, donde $(1, 1)$ activa la corrección):

Época	w_1 antes	w_2 antes	Ejemplo que falla	$z = w_1x_1 + w_2x_2$	y (con $\theta = 1.5$)	t	$\Delta w_1 = \alpha(t-y)x_1$	$\Delta w_2 = \alpha(t-y)x_2$	w_1 después	w_2 después
1	0.00	0.25	$(1,1)$	0.25	0	1	0.3	0.3	**0.30**	**0.55**
2	0.30	0.55	$(1,1)$	0.85	0	1	0.3	0.3	**0.60**	**0.85**
3	0.60	0.85	$(1,1)$	1.45	0	1	0.3	0.3	**0.90**	**1.15**
4	0.90	1.15	— (ningún ejemplo falla: $(1,1) \to z = 2.05 \geq 1.5 \to y = 1 = t$; los demás ya daban $y = 0 = t$)						**se detiene**	

Pesos finales: $w_1 = 0.9,\ w_2 = 1.15$, con los cuales el perceptrón clasifica correctamente las 4 combinaciones de AND. Nótese que $w_1 \neq w_2$ en la solución final aun cuando el problema es simétrico en x_1, x_2 — otro ejemplo de **no unicidad de la solución**: cualquier par de pesos que satisfaga $w_1, w_2 < 1.5$ y $w_1 + w_2 \geq 1.5$ es una solución válida.

🔁 **Errores comunes en este tipo de ejercicio:**

- Detener el entrenamiento en la primera época donde *algún* ejemplo se clasifica bien, sin verificar **todos** los ejemplos de esa época.
- Olvidar que el bias, aunque se omita por simplicidad en el ejercicio, "siempre está" en la formulación general.
- Confundir el criterio de parada: no es "el error llegó a cero en un ejemplo", sino "el error es cero para **todos** los ejemplos simultáneamente".

12.1 Definición formal

Backpropagation (retropropagación del error) es el algoritmo que calcula, de manera eficiente, el gradiente de la función de error respecto de **cada parámetro** de la red, aplicando sistemáticamente la **regla de la cadena** desde la capa de salida hacia la capa de entrada. Goodfellow enfatiza que backpropagation no es "otro algoritmo de aprendizaje" — es únicamente el método de **cálculo del gradiente**; el aprendizaje propiamente dicho lo realiza gradiente descendente (u otro optimizador) usando ese gradiente.

Por qué es necesario: en una red con muchas capas, calcular ingenuamente la derivada de J respecto de cada peso mediante la regla de la cadena "extendida" (multiplicando explícitamente todas las derivadas parciales desde la salida hasta ese peso) recalcularía las mismas subexpresiones una y otra vez. Backpropagation reorganiza este cálculo en un **grafo computacional**, reutilizando resultados intermedios y logrando que el costo total de calcular todos los gradientes sea del mismo orden que un único forward pass.

12.2 La regla de la cadena aplicada a una red

Para un peso arbitrario w de la capa l, el gradiente de la función de error J se descompone como:

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Este es el patrón fundamental repetido en cada capa: **error propagado hasta la salida de la neurona × derivada de la activación × derivada de la pre-activación respecto del peso.**

12.3 Derivación completa, capa por capa

Sea una MLP de L capas, con función de error total J. Se define el **término de error local** (o *delta* de backpropagation, distinto de la regla delta de §11 pero relacionado) de la neurona j en la capa l:

$$\delta_j^{(l)} \equiv \frac{\partial J}{\partial z_j^{(l)}}$$

Paso 1 — Inicialización del gradiente en la capa de salida (⚠ NIVEL EXAMEN — el profesor lo señala explícitamente, incluso presente en arquitecturas Transformer):

$$\delta_j^{(L)} = \frac{\partial J}{\partial a_j^{(L)}} \cdot f_L'\left(z_j^{(L)}\right)$$

Es decir: se calcula primero **la derivada de la divergencia respecto de la salida de la red**, y se multiplica por la derivada local de la activación de la última capa. (Como se vio en §10.3,

si la salida usa softmax y la pérdida es entropía cruzada, esta expresión colapsa elegantemente a $\delta^{(L)} = \hat{\mathbf{y}} - \mathbf{d}$.)

Paso 2 — Propagación recursiva hacia atrás, para $l = L-1, \ldots, 1$:

$$\delta_j^{(l)} = \left(\sum_k w_{jk}^{(l+1)} \, \delta_k^{(l+1)} \right) \cdot f_l'\left(z_j^{(l)}\right)$$

Esta ecuación es el corazón de backpropagation: el error de la neurona j en la capa l es una **suma ponderada de los errores de todas las neuronas de la capa siguiente a las que j está conectada**, ponderada exactamente por los **mismos pesos** que se usaron en el forward pass — de ahí el nombre "retro-propagación": el error viaja hacia atrás por los mismos caminos que la señal viajó hacia adelante, en sentido inverso.

Observación notacional clave (Anotaciones, Clase 10): dado que $z^{(l+1)} = W^{(l+1)} y^{(l)}$, se tiene que $\dfrac{\partial z^{(l+1)}}{\partial y^{(l)}} = W^{(l+1)}$ — es decir, **la derivada de la pre-activación de una capa respecto de la salida de la capa anterior es, literalmente, la matriz de pesos**. Esta identidad es la que permite escribir el paso 2 de forma compacta en notación vectorial:

$$\boldsymbol{\delta}^{(l)} = \left(\left(W^{(l+1)}\right)^T \boldsymbol{\delta}^{(l+1)} \right) \odot f_l'\left(\mathbf{z}^{(l)}\right)$$

donde $\odot$ denota el producto elemento a elemento (Hadamard).

Paso 3 — Gradiente respecto de los pesos y el bias:

$$\frac{\partial J}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} \cdot y_i^{(l-1)} \qquad\qquad \frac{\partial J}{\partial b_j^{(l)}} = \delta_j^{(l)}$$

En forma matricial: $\dfrac{\partial J}{\partial W^{(l)}} = \boldsymbol{\delta}^{(l)} \left(\mathbf{a}^{(l-1)}\right)^T$.

Paso 4 — Actualización de pesos (gradiente descendente):

$$w_{ij}^{(l)} := w_{ij}^{(l)} - \frac{\alpha}{M} \frac{\partial \, \text{Error}}{\partial w_{ij}^{(l)}}$$

donde M es el tamaño del lote (*batch*) usado para promediar el gradiente, y α la tasa de aprendizaje.

12.4 Algoritmo completo de backpropagation (resumen operativo)

Entrada: salida deseada $\mathbf{d}$, pesos actuales, función de divergencia div, activaciones guardadas del forward pass. **Salida:** gradientes $\partial J / \partial w$ para cada peso de la red.

1. **Forward pass:** calcular y **guardar** todos los $z_j^{(l)}$ y $y_j^{(l)}$ de cada capa (son necesarios para el backward — no guardar activaciones es un error común señalado explícitamente en las notas).
2. Calcular el error/divergencia entre $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$ y $\mathbf{d}$.
3. **Inicializar el gradiente** en la capa de salida: $\boldsymbol{\delta}^{(L)}$ (Paso 1 arriba).
4. **Bucle hacia atrás**, para $l = L-1$ hasta 1: calcular $\boldsymbol{\delta}^{(l)}$ (Paso 2), y de inmediato $\partial J/\partial W^{(l)}, \partial J/\partial b^{(l)}$ (Paso 3).
5. **Actualizar** todos los pesos con gradiente descendente (Paso 4).
6. **Criterio de parada:** convergencia del error total sobre el conjunto de entrenamiento (nunca basta con una sola época: *"una época nunca es suficiente para que el error converja"*).

12.5 Ejemplo numérico resuelto: un paso de backpropagation completo

Enunciado. Red de una entrada, una capa oculta con una neurona sigmoide, una salida lineal. Pesos: $w^{(1)} = 0.5$ (entrada→oculta, sin bias por simplicidad), $w^{(2)} = 1.2$ (oculta→salida). Entrada $x = 1$, salida deseada $d = 1$. Función de error: MSE $J = \frac{1}{2}(\hat{y} - d)^2$. Tasa de aprendizaje $\alpha = 0.5$. Calcular el gradiente y actualizar ambos pesos.

Solución.

Forward:

$$z^{(1)} = w^{(1)}x = 0.5(1) = 0.5 \qquad a^{(1)} = \sigma(0.5) = \frac{1}{1+e^{-0.5}} \approx 0.6225$$

$$z^{(2)} = w^{(2)}a^{(1)} = 1.2(0.6225) \approx 0.7470 \qquad \hat{y} = z^{(2)} = 0.7470 \quad \text{(activación lineal)}$$

Error: $J = \frac{1}{2}(0.7470 - 1)^2 \approx 0.0320$.

Backward — inicialización del gradiente en la salida:

$$\delta^{(2)} = \frac{\partial J}{\partial \hat{y}} \cdot f'_{\text{lineal}}(z^{(2)}) = (\hat{y} - d) \cdot 1 = 0.7470 - 1 = -0.2530$$

Gradiente del peso de salida:

$$\frac{\partial J}{\partial w^{(2)}} = \delta^{(2)} \cdot a^{(1)} = (-0.2530)(0.6225) \approx -0.1575$$

Propagación al peso oculto:

$$\delta^{(1)} = w^{(2)}\delta^{(2)} \cdot f'_{\sigma}(z^{(1)}) = (1.2)(-0.2530) \cdot [\sigma(0.5)(1-\sigma(0.5))]$$

$$\sigma(0.5)(1-\sigma(0.5)) = 0.6225(1-0.6225) = 0.6225(0.3775) \approx 0.2350$$

$$\delta^{(1)} = (1.2)(-0.2530)(0.2350) \approx -0.0714$$

$$\frac{\partial J}{\partial w^{(1)}} = \delta^{(1)} \cdot x = (-0.0714)(1) = -0.0714$$

Actualización de pesos:

$$w^{(2)}_{\text{nuevo}} = 1.2 - 0.5(-0.1575) = 1.2 + 0.0788 = 1.2788$$

$$w^{(1)}_{\text{nuevo}} = 0.5 - 0.5(-0.0714) = 0.5 + 0.0357 = 0.5357$$

Ambos pesos aumentaron, lo cual es consistente con la intuición: $\hat{y} < d$ (la predicción fue insuficiente), por lo que la corrección **incrementa** los pesos que contribuyeron positivamente a la salida.

12.6 Errores comunes en backpropagation (banco de exámenes)

- Creer que solo se corrige la última capa — **todos** los pesos de **todas** las capas contribuyen al error final y deben actualizarse.
- No guardar las activaciones del forward pass (son indispensables para calcular δ en el backward).
- Usar una activación no diferenciable (escalón) en una red que se entrena con backpropagation estándar.
- Olvidar el bias en la actualización.
- Confundir la función de costo (que se mide *fuera* de la arquitectura) con la arquitectura misma.
- No inicializar correctamente el gradiente en la capa de salida (paso más frecuentemente señalado como "nivel examen").

---

13. Notación vectorial general y el jacobiano

13.1 Tensores: generalización de escalar, vector y matriz

Objeto	Orden (rango)	Ejemplo
Escalar	0	7

Objeto	Orden (rango)	Ejemplo
Vector	1	$[1, 2, 3]$
Matriz	2	Imagen en escala de grises ($H \times W$)
Tensor 3D	3	Imagen RGB ($H \times W \times C$)
Tensor 4D	4	Lote (*batch*) de imágenes ($N \times H \times W \times C$)

Incluso una entrada escalar se representa formalmente como un vector de una componente, para mantener consistencia en la notación matricial de toda la red.

13.2 El jacobiano

Definición formal. Si $f : \mathbb{R}^n \to \mathbb{R}^m$, el jacobiano de f es la matriz de todas las derivadas parciales de cada componente de salida respecto de cada componente de entrada:

$$J_f = \begin{bmatrix} \dfrac{\partial f_1}{\partial x_1} & \cdots & \dfrac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial f_m}{\partial x_1} & \cdots & \dfrac{\partial f_m}{\partial x_n} \end{bmatrix} \in \mathbb{R}^{m\times n}$$

Intuición: mide cuánto cambia **cada componente** de la salida ante una pequeña perturbación en **cada componente** de la entrada. Es la generalización multivariable del concepto de derivada / gradiente.

⚠ **Distinción de examen:** el **gradiente** se define para funciones de salida **escalar** ($f : \mathbb{R}^n \to \mathbb{R}$) y es un vector $\nabla f \in \mathbb{R}^n$. El **jacobiano** se define para funciones de salida **vectorial** ($f : \mathbb{R}^n \to \mathbb{R}^m, m > 1$) y es una matriz. El gradiente es, de hecho, un caso particular del jacobiano cuando $m = 1$ (una matriz $1 \times n$, transpuesta por convención a vector columna).

Jacobiano diagonal vs. denso: si la función aplica una transformación **elemento a elemento** (como la sigmoide o ReLU aplicadas a cada componente independientemente), su jacobiano es **diagonal** — cada salida depende únicamente de la entrada en la misma posición. Si la función mezcla las componentes (como una capa densa $W\mathbf{x}$, o softmax), el jacobiano es en general **denso** (todos contra todos).

13.3 Aplicación: cómo medir perturbaciones entrada→salida en una red profunda

Pregunta de examen (respuesta modelo). *"¿Cómo es posible medir las perturbaciones en la salida de una red neuronal cuando se producen pequeñas perturbaciones en la entrada x ?"* Respuesta correcta: **midiendo las pequeñas perturbaciones que cada función genera individualmente en el camino desde la entrada hasta la salida**, es decir, componiendo (multiplicando) los jacobianos locales de cada capa mediante la regla de la cadena — exactamente el mismo mecanismo que sustenta backpropagation, pero en dirección hacia adelante (sensibilidad) en lugar de hacia atrás (gradiente del error).

13.4 Rol computacional

La disponibilidad de bibliotecas de álgebra lineal vectorizada (NumPy y equivalentes) hizo computacionalmente viable calcular jacobianos y gradientes de redes con millones de parámetros — el profesor lo señala como uno de los "tres motores" que hicieron posible el auge práctico de las redes neuronales (junto con la disponibilidad de datos y la capacidad de cómputo).

---

14. Estrategias de actualización: online (SGD), batch, mini-batch

Estrategia	Descripción	Ventaja	Desventaja
Online / SGD (*Stochastic Gradient Descent*)	Se actualiza el peso después de **cada ejemplo individual**, presentado en orden aleatorio	Actualización rápida, útil cuando hay redundancia en los datos	Ruidoso; **no siempre conveniente** — por ejemplo, en datos secuenciales donde el **orden** de presentación importa, aleatorizar destruye información relevante
Batch	Se calcula el gradiente promediando **todos** los ejemplos del conjunto de entrenamiento antes de actualizar	Gradiente estable, dirección precisa	Muy lento por iteración; requiere mantener todo el dataset en memoria
Mini-batch	Se promedia el gradiente sobre **subconjuntos** (lotes) del conjunto de entrenamiento	Compromiso entre velocidad y estabilidad; es el estándar en la práctica	Introduce un hiperparámetro adicional (tamaño del lote M)

Fórmula general de actualización con normalización por lote:

$$w_{ij}^{(k)} := w_{ij}^{(k)} - \frac{\alpha}{M} \sum_{m=1}^{M} \frac{\partial \, \text{Error}_m}{\partial w_{ij}^{(k)}}$$

Mínimos locales. El riesgo de quedar atrapado en un mínimo local (o en una meseta/*saddle point*) es **mayor en redes neuronales** que en modelos lineales simples, precisamente por la enorme cantidad de conexiones y por la no convexidad de la función de error resultante de componer funciones no lineales. Estrategias mitigantes:

- Controlar cuidadosamente el **punto de inicialización** de los pesos (aleatoria, normalizada, controlada).

- **Transfer Learning:** iniciar desde pesos ya entrenados en una tarea relacionada, en vez de partir de cero.
- Fijar el ***seed*** aleatorio para poder **replicar** exactamente un experimento (relevante también por la no unicidad de los parámetros, §6.5).

🔁 **Trampa recurrente (Pregunta 87, 95):**

- **No** es cierto que el gradiente descendente, con suficientes iteraciones, siempre encuentre el mínimo **global** — puede quedar atrapado en mínimos locales o puntos estacionarios.
- Si w_0, w_1 se inicializan **exactamente en el mínimo global**, **una iteración de gradiente descendente no cambiará sus valores** (el gradiente ahí es cero) — esto **sí** es verdadero.
- Que $w_0 = w_1$ al inicio **no** garantiza simetría después de una iteración, salvo que la función de costo sea simétrica respecto de ambos parámetros de forma exacta — en general, **no** se puede afirmar que $w_0 = w_1$ se preserve.
- Un valor de α (tasa de aprendizaje) demasiado grande **puede hacer que el costo aumente** en vez de disminuir en cada iteración — la afirmación "toda iteración de GD disminuye J sin importar α" es **falsa**.

---

15. Regularización en MLP: overfitting, dropout, L1/L2

15.1 Overfitting y underfitting (repaso aplicado a MLP)

- **Overfitting (sobreajuste):** el modelo **memoriza** los ejemplos de entrenamiento (incluido su ruido) en vez de **aprender** el patrón subyacente; se manifiesta como error de entrenamiento muy bajo pero error de validación/prueba alto. Analogía explícita del curso: es equivalente al riesgo de una regresión polinómica de grado demasiado alto.
- **Underfitting:** el modelo es demasiado simple (pocas neuronas, pocas capas) para capturar el patrón; error alto tanto en entrenamiento como en validación.
- **Factor que más afecta ambos problemas en una red neuronal:** el **número de nodos/neuronas ocultas** (la capacidad del modelo) — no tanto los pesos iniciales, ni la tasa de aprendizaje, ni el tamaño de los lotes (Pregunta 57).

Soluciones, en el orden de preferencia enfatizado por el profesor:

1. **Más ejemplos de entrenamiento** — repetidamente señalada como *"la mejor solución"* ("mientras más ejemplos, más precisos los pesos").
2. Reducir el número de características/parámetros (selección manual, PCA).
3. Añadir un término de **regularización** (L1 o L2) a la función de costo.
4. Aplicar **dropout** (específico de redes neuronales, ver §15.3).

5. Normalización de los datos de entrada y **validación cruzada** — controlan/detectan el overfitting, pero no lo "resuelven" por sí solas.

🔁 **Trampa (Preguntas 58, 86):**

- Los conjuntos de entrenamiento **más grandes** son **menos** propensos a producir overfitting (no más — la afirmación contraria es falsa).
- Un número **grande** de características (*features*) **sí** es más probable que produzca overfitting (verdadero).
- A medida que el número de ejemplos de entrenamiento crece hacia infinito, la probabilidad de que la regresión logística sobreajuste **se reduce a cero** (verdadero — refleja el mismo principio de "más ejemplos, mejor").
- Si una red tiene overfitting, la solución es **aumentar** el parámetro de regularización λ (no disminuirlo), y **reducir** la complejidad (no aumentar el número de capas).

15.2 Regularización L1 y L2

$$J_{\text{L1}}(\mathbf{w}) = J(\mathbf{w}) + \lambda \sum_j |w_j| \qquad J_{\text{L2}}(\mathbf{w}) = J(\mathbf{w}) + \lambda \sum_j w_j^2$$

- **L1 (Lasso):** penaliza el valor absoluto de los pesos; tiende a llevar algunos pesos exactamente a **cero**, realizando una forma implícita de selección de características (*"elimina extremos"*).
- **L2 (Ridge):** penaliza el cuadrado de los pesos; **suaviza** los pesos sin anularlos (*"suaviza, no elimina"*).
- λ **grande** → mayor penalización → modelo más simple (riesgo de underfitting, "todo tiende a una recta"). λ **pequeño** → menor penalización → modelo más flexible (riesgo de overfitting).

15.3 Dropout

⚠ **NIVEL EXAMEN, marcado explícitamente por el profesor.**

Definición formal. Dropout es una técnica de regularización específica de redes neuronales que, **durante el entrenamiento únicamente**, desactiva aleatoriamente una fracción $p \in (0, 1)$ de las neuronas de cada capa oculta en cada paso de entrenamiento (paso *forward*), forzando a la red a no depender excesivamente de neuronas o combinaciones de neuronas específicas.

$$\text{neuronas activas} = 1 - p \qquad \text{neuronas apagadas} = p$$

🔁 **Trampa explícita (Pregunta 65, y remarcada por el profesor como "tema examen"):** si el dropout apaga el $p\%$ de las neuronas, la fracción que permanece **activa** es $1 - p$ — **no** p.

Es un error muy frecuente confundir cuál fracción corresponde a "encendidas" y cuál a "apagadas".

Otras propiedades verdaderas sobre dropout (Pregunta 65):

- Durante el entrenamiento se **mantienen** los nodos con probabilidad $1 - p$ (verdadero — reformulación de lo anterior).
- Es **falso** que la técnica de dropout implique que las capas se "adapten para corregir errores de capas posteriores" — esa descripción corresponde más bien a la intuición de *boosting*, no a dropout.
- Dropout se aplica en el paso **forward** durante el entrenamiento; en inferencia (evaluación/producción) **todas** las neuronas están activas, típicamente reescalando las activaciones para compensar (detalle de implementación, mencionado por completitud).

15.4 Validación cruzada y fragmentación del conjunto de datos

Por qué se fragmentan los datos (Pregunta 63):

- Los modelos necesitan datos separados para **entrenar** y para **validar**; sin un conjunto de validación no hay garantía de que el modelo generalice (no de que "el error converja a cero" — el error de entrenamiento puede converger sin que el modelo generalice).
- Modelos con **muchos parámetros** (como las MLP) requieren especialmente ser evaluados con **validación cruzada**, dado su alto riesgo de overfitting.
- La validación cruzada **controla/detecta** el overfitting; **no lo resuelve por sí sola** — la solución de fondo sigue siendo alguna de las estrategias de §15.1.

Validación cruzada n-fold: cada dato pertenece exactamente a un subconjunto de prueba (los subconjuntos de prueba son independientes entre sí). Sin embargo, **las estimaciones de error de los distintos folds NO son independientes entre sí** (Pregunta 98: la respuesta correcta es **Falso**), porque los datos de entrenamiento se solapan parcialmente entre folds — cada fold comparte la mayoría de sus datos de entrenamiento con los demás folds, introduciendo correlación entre las estimaciones de error.

---

16. Banco de problemas resueltos (estilo examen)

Esta sección resuelve, con desarrollo formal completo, las variantes de preguntas de MLP/perceptrón más representativas del banco histórico (`Preguntas_de_IA.pdf`). Se agrupan por tipo de habilidad evaluada.

16.1 Conteo de parámetros en capas densas

Problema tipo (Preguntas 12, 30, 91, 140). Imagen a color 500×500 (sin CNN), primera capa oculta con 50 neuronas totalmente conectadas. ¿Cuántos parámetros tiene esa capa?

Solución. Entradas aplanadas: $500 \times 500 \times 3 = 750{,}000$. Parámetros $= 50 \times (750{,}000 + 1) = 50 \times 750{,}001 = 37{,}500{,}050$.

Regla general memorizable: para imagen $N \times N \times 3$ con h neuronas en la primera capa densa:

$$\#\text{parámetros} = h\,(3N^2 + 1)$$

16.2 Dimensiones de matrices de pesos en redes con varias capas

Problema tipo (Pregunta 139). Red de 4 capas de neuronas: capa 1 (entrada) con 2, capa 2 con 4, capa 3 con 5, capa 4 (salida) con 3. Hallar la dimensión de $[W^{(2)}|W^{(3)}]$ (concatenación de las matrices de la transición 2→3 y 3→4).

Solución. Usando la convención $W^{(k)} = S_{k+1} \times (S_k + 1)$ donde S_k es el número de neuronas de la capa k:

$$W^{(2)} = S_3 \times (S_2 + 1) = 5 \times (3 + 1) = 5 \times 4$$

$$W^{(3)} = S_4 \times (S_3 + 1) = 3 \times (5 + 1) = 3 \times 6$$

(Nota: en este ejercicio concreto del banco, la numeración de capas usada difiere ligeramente entre enunciado y figura; **la estrategia correcta siempre es identificar, mirando la figura, cuántas neuronas hay en la capa origen y cuántas en la capa destino de cada matriz nombrada**, y aplicar $\text{destino} \times (\text{origen} + 1)$.)

16.3 Construcción de funciones booleanas con neuronas

Problema tipo (Pregunta 68, 112). ¿Es posible representar con una sola neurona (activación logística/hardlim) la función:

X_1	X_2	f
1	1	0
0	0	0
1	0	1
0	1	0

Solución. Esta función vale 1 únicamente en $(1, 0)$: equivale a $X_1 \wedge \neg X_2$, que **sí es linealmente separable** (un solo punto positivo, fácilmente aislable con una recta). Con función hardlim centrada en cero (umbral en 0) y pesos $w_{01} = -0.5,\ w_{11} = 1,\ w_{21} = -1$:

$$z = -0.5 + 1 \cdot X_1 - 1 \cdot X_2$$

Verificación: $(1,1) \to z = -0.5 < 0 \to 0 \checkmark$; $(0,0) \to z = -0.5 < 0 \to 0 \checkmark$; $(1,0) \to z = 0.5 \geq 0 \to 1 \checkmark$; $(0,1) \to z = -1.5 < 0 \to 0 \checkmark$. **Sí es posible con una sola neurona.**

Contraste (Pregunta 68, tabla distinta): si la función a representar fuera XOR-like (activa en $(1,0)$ **y** $(0,1)$, inactiva en $(0,0)$ y $(1,1)$), **no** sería posible con una sola neurona, por el mismo argumento formal de no separabilidad lineal demostrado en §4.1.

16.4 Compuertas booleanas complejas con red de 2 capas

Problema tipo (Preguntas 88, 97). Red con 2 entradas binarias, salida sigmoidal, pesos de entrada $[30, -20, -20]$ (bias, w_1, w_2). ¿Qué función lógica implementa?

Solución. $z = 30 - 20x_1 - 20x_2$. Evaluando: $(0,0) \to z = 30 \to \sigma(30) \approx 1$; $(0,1) \to z = 10 \to \sigma(10) \approx 1$; $(1,0) \to z = 10 \to \approx 1$; $(1,1) \to z = -10 \to \approx 0$. Esto corresponde exactamente a **NAND** (la única combinación que da 0 es $(1,1)$).

16.5 Regla delta / actualización de gradiente en regresión logística

Problema tipo (Pregunta 29). Dado el gradiente de la regresión logística

$$\frac{\partial}{\partial w_j} J(w) = \frac{1}{m} \sum_{i=1}^{m} \left(h_w(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

¿cuál es la actualización correcta de gradiente descendente?

Solución. La actualización correcta es

$$w_j := w_j - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(h_w(x^{(i)}) - y^{(i)} \right) x_j^{(i)} \qquad \text{(actualizar simultáneamente para todos los } j\text{)}$$

🔁 **Trampa:** una variante incorrecta muy común en el banco sustituye $h_w(x^{(i)})$ por $w^T x^{(i)}$ (la pre-activación **sin** pasar por la sigmoide) — esto sería válido para regresión **lineal**, no para regresión **logística**, donde $h_w(x) = \sigma(w^T x)$ es indispensable. También es incorrecto escribir explícitamente $\frac{1}{1+e^{-w^t x}}$ y luego **no** restar $y^{(i)}$ correctamente, o alterar el orden de la resta.

16.6 Interpretación probabilística de la salida logística

Problema tipo (Pregunta 59). Si $h_w(x) = 0.4$ para un nuevo ejemplo, ¿qué se puede afirmar?

Solución. Por definición, $h_w(x) = P(y = 1 \mid x; w)$. Entonces:

- $P(y = 1 \mid x; w) = 0.4 \checkmark$ (verdadero)

- $P(y = 0 \mid x; w) = 1 - 0.4 = 0.6$ ✓ (verdadero, por complementariedad)
- Es **falso** decir que $P(y = 0 \mid x; w) = 0.4$ o que $P(y = 1 \mid x; w) = 0.6$ — son las afirmaciones invertidas.

16.7 Fronteras de decisión a partir de pesos

Problema tipo (Preguntas 136, 150). Clasificador $h_w(x) = g(w_0 + w_1 x_1 + w_2 x_2)$ con $w_0 = -6,\ w_1 = 1,\ w_2 = 0$. Hallar la frontera de decisión.

Solución. La frontera ocurre donde $w_0 + w_1 x_1 + w_2 x_2 = 0$ (equivalente a $h_w(x) = 0.5$):

$$-6 + x_1 = 0 \implies x_1 = 6$$

Como $w_2 = 0$, la frontera **no depende de** x_2: es una recta **vertical** en $x_1 = 6$. Para $x_1 > 6$, $z > 0 \Rightarrow y = 1$; para $x_1 < 6, y = 0$. 🔁 **Trampa:** confundir el sentido de la desigualdad (cuál lado es $y = 1$) es el error más frecuente — siempre verificar sustituyendo un punto de prueba concreto (p. ej. $x_1 = 10$: $z = 4 > 0 \rightarrow y = 1$, confirmando que la región **derecha** de la recta es $y = 1$).

16.8 Convexidad y unicidad de la solución en regresión logística

Problema tipo (Pregunta 56). Si se encuentra $J(w_0, w_1) = 0$ para el conjunto de entrenamiento, ¿qué se puede afirmar?

Solución. $J = 0$ implica que $h_w(x^{(i)}) = y^{(i)}$ **exactamente** para cada ejemplo de entrenamiento (verdadero). **No** implica, sin embargo, que el modelo prediga perfectamente ejemplos **nuevos** (no vistos) — eso sería una conclusión de generalización, que $J = 0$ en entrenamiento **no garantiza** (de hecho, $J = 0$ exacto en entrenamiento suele ser señal de overfitting severo).

16.9 Aplicabilidad de redes neuronales a distintos paradigmas de aprendizaje

Problema tipo (Pregunta 89). ¿Se puede entrenar una red neuronal para jugar ajedrez con cada paradigma de aprendizaje?

Solución razonada:

- **Supervisado:** sí — representando posiciones del tablero como entrada y una etiqueta (p. ej. "mejor movimiento" o "resultado de la partida") como salida, usando partidas históricas como ejemplos de entrenamiento.
- **No supervisado: no** tiene sentido directo para jugar (no hay una variable dependiente que optimizar para "ganar"), aunque sí podría usarse no-supervisadamente para tareas auxiliares como agrupar posiciones similares — pero **no** para aprender la estrategia de juego en sí.
- **Por refuerzo:** sí, y es de hecho el paradigma más natural — el agente prueba estrategias, recibe recompensa (ganar) o penalización (perder), y ajusta su política en consecuencia (este es el principio detrás de sistemas como AlphaZero).

16.10 Perturbación de pesos y equivalencia funcional

Problema tipo (Pregunta 61). Si se intercambian simultáneamente los pesos de dos neuronas ocultas (tanto en $W^{(1)}$ como en $W^{(2)}$, de forma consistente), ¿cambia la salida $h_w(x)$?

Solución. No cambia — es una aplicación directa del principio de no unicidad por permutación (§6.5): intercambiar dos neuronas ocultas junto con sus conexiones de entrada y de salida asociadas es una simple relabelización interna que no altera la función calculada por la red.

---

17. Tabla-resumen de fórmulas críticas

Concepto	Fórmula
Neurona / perceptrón	$z = \mathbf{W}^T\mathbf{X}, a = f(z)$
Dimensión de $W^{(k)}$	$\mathbb{R}^{L_{k+1}\times(L_k+1)}$
Forward por capa	$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, a^{(l)} = f^{(l)}(z^{(l)})$
Sigmoide y derivada	$\sigma(z) = \dfrac{1}{1+e^{-z}}; \sigma'(z) = \sigma(z)(1-\sigma(z))$
Tanh y derivada	$\tanh(z); 1 - \tanh^2(z)$
ReLU y derivada	$\max(0, z); 1 \text{ si } z > 0, 0 \text{ si } z < 0$
Softmax	$\text{softmax}(z)_k = \dfrac{e^{z_k}}{\sum_j e^{z_j}}$
Regla delta	$\Delta w_{ij} = \alpha(t_j - y_j)y_i$
Inicialización del gradiente (salida)	$\delta^{(L)} = \dfrac{\partial J}{\partial a^{(L)}} \odot f_L'(z^{(L)})$
Propagación del error hacia atrás	$\delta^{(l)} = \left((W^{(l+1)})^T\delta^{(l+1)}\right) \odot f_l'(z^{(l)})$
Gradiente respecto de pesos	$\partial J/\partial w_{ij}^{(l)} = \delta_j^{(l)} y_i^{(l-1)}$
Softmax + entropía cruzada (atajo)	$\partial J/\partial z_k = \hat{y}_k - d_k$
Actualización de pesos	$w := w - \dfrac{\alpha}{M}\partial J/\partial w$
Regularización L1 / L2	$+\lambda\sum
Dropout	neuronas activas $= 1 - p$
Jacobiano	$J_f \in \mathbb{R}^{m\times n}, (J_f)_{kl} = \partial f_k/\partial x_l$

---

18. Checklist final de dominio

- [ ] Puedo explicar la analogía neurona biológica ↔ perceptrón (dendritas/sinapsis/soma/axón).
- [ ] Puedo construir compuertas AND/OR/NOT con un perceptrón, indicando pesos y umbral explícitos.
- [ ] Puedo demostrar formalmente por qué XOR no es linealmente separable.
- [ ] Puedo construir una MLP que resuelva XOR/XNOR indicando todos los pesos.
- [ ] Puedo calcular la dimensión de cualquier matriz de pesos $W^{(k)}$, incluyendo el bias.
- [ ] Puedo contar el número total de parámetros de una MLP dada su arquitectura.
- [ ] Puedo ejecutar manualmente un forward pass completo dada una arquitectura y pesos concretos.
- [ ] Puedo aplicar la regla delta y entrenar un perceptrón por épocas hasta convergencia.
- [ ] Puedo derivar y aplicar backpropagation paso a paso: inicialización del gradiente en la salida, propagación hacia atrás, cálculo de $\partial J / \partial w$, actualización.
- [ ] Puedo explicar por qué las funciones de activación deben ser diferenciables para backpropagation.
- [ ] Puedo distinguir cuándo usar x vs. y en la regla delta / backpropagation según la capa de origen.
- [ ] Puedo explicar softmax, sus propiedades, y por qué no se usa en capas ocultas.
- [ ] Puedo comparar funciones de divergencia (MSE, entropía, entropía cruzada) y justificar cuándo usar cada una.
- [ ] Puedo explicar la inicialización del gradiente y por qué comienza en la capa de salida.
- [ ] Puedo definir el jacobiano, diferenciarlo del gradiente, y explicar cuándo es diagonal.
- [ ] Puedo comparar SGD online, batch y mini-batch.
- [ ] Puedo explicar dropout con precisión (activas $= 1 - p$) y su efecto regularizador.
- [ ] Puedo explicar L1 vs. L2 y el efecto de λ.
- [ ] Puedo diagnosticar overfitting/underfitting y ordenar las soluciones según su prioridad práctica.
- [ ] Puedo razonar sobre la no unicidad de los parámetros (permutación de neuronas ocultas).
- [ ] Puedo hallar la frontera de decisión de un clasificador logístico/perceptrón dado un vector de pesos.

Fuentes utilizadas

- Anotaciones normalizadas de clase (Clases 06–12): regresión logística como puente,

perceptrón, MLP, XOR/XNOR, backpropagation, softmax, jacobiano, dropout.

- Guía maestra del curso — secciones 8 a 15 (MLP, limitaciones del perceptrón, aproximador universal, forward/backpropagation, regla delta, jacobiano y tensores).
- Banco de preguntas de exámenes anteriores (`Preguntas_de_IA.pdf`) — preguntas 10–13, 27–30, 57, 59–62, 65, 68, 71, 87–91, 95–98, 112–115, 120, 133–137, 139–150.
- Goodfellow, I., Bengio, Y., Courville, A. — *Deep Learning* (formalización de backpropagation, grafos computacionales, regularización).
- Hagan, M. et al. — *Neural Network Design* (funciones de transferencia, regla delta / perceptron learning rule).
- Brownlee, J. — *Basic Machine Learning* (gradiente descendente, fundamentos de regresión aplicados como base de la neurona).
- Luger, G. — *Artificial Intelligence: Structures and Strategies* (contexto de IA simbólica vs. conexionista).